Lecture 8: Fault-Tolerance w/ Replicated State Machines (RSMs)

CS 739: Distributed Systems

Prof. Andrea C. Arpaci-Dusseau
University of Wisconsin — Madison

Schedule of Topics: Projects

Congrats on finishing Project 1!

Project 2 Available: Build from P1

- Add durability and partition key space across servers
- Can switch project partners
- Due < 3 weeks – Sunday 3/8

P3: Due (w/ demo) before Spring Break – Fri 3/25

- Add replication for fault-tolerance
- More challenging than P1 or P2

Open-Ended Project – Due Thu 5/7 – Posted Final Exam day

Schedule of Topics: Midterm Exam

This week: Fault-tolerance w/ replication

- RSM + Primary/Backup (HA-NFS + Remus)

Next week (2/25 + 2/27): Miscellaneous (work on P2!)

- Serverless Computing (Tingjia) + P2 overview (Junxuan)

3/4 + 3/6: Fault-tolerance + Consensus

- Chain Replication + Paxos (P2 due Sunday 3/8)

3/11 + 3/3: Two In-class Midterm Exams

- Wed: Assess ability to independently read + understand (new to you) paper
  - Given surprise ”traditional distributed systems paper” that doesn’t rely on topics from later in semester
  - Can refer to paper while answering questions
  - Multiple-choice questions will be straight-forward and follow order of paper
  - Requires basic understanding of covered 739 content (no explicit comparisons expected)
  - Low stakes: Worth only 15% of midterm grade
  - Repeat experiment for Final Exam
- Fri: Multiple-choice questions over
  - Closed notes; can bring 1 double-sided paper with notes
  - Covers all materials through exam (including Paxos)
  - Many sample questions posted in Canvas

4/29 + 5/1: Two In-Class Final Exams – Paper Reading and Multiple-Choice

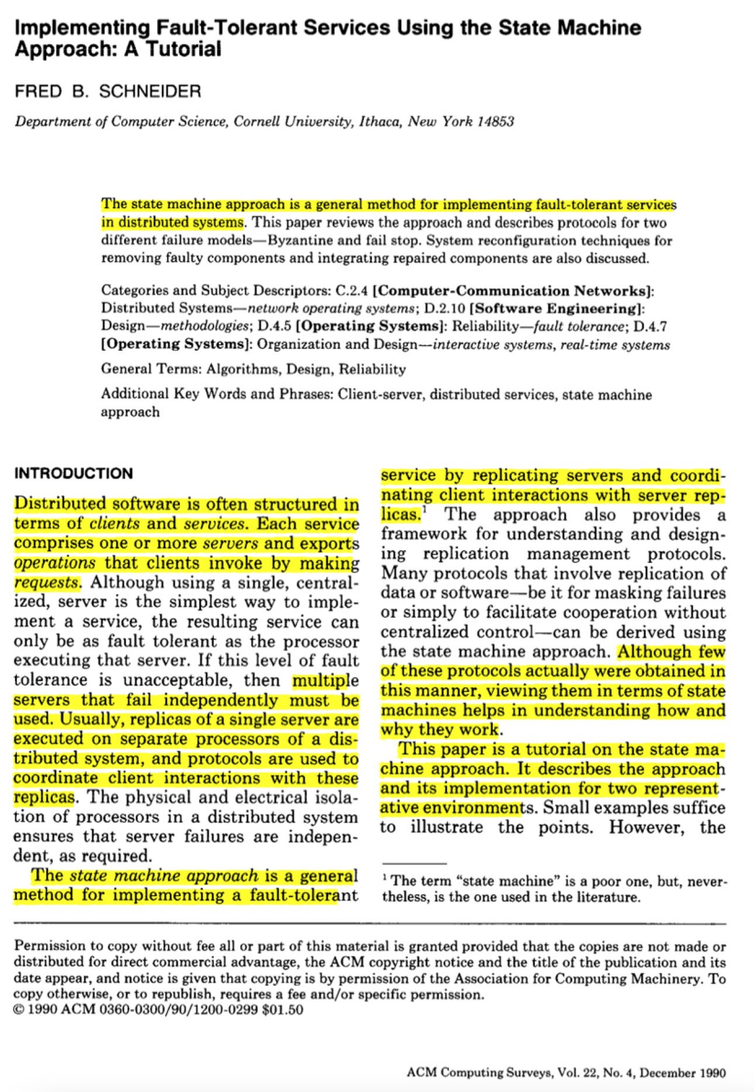
Today's Topic: Replicated State Machines

Fred B. Schneider, Implementing Fault-Tolerant Services Using the State Machine Approach: a tutorial, ACM Computing Surveys 22(4):299-319, December 1990. The paper that explained how we should think about replication … a model that turns out to underlie Paxos, Virtual Synchrony, Byzantine replication, and even Transactional 1-Copy Serializability.

General framework of describing a distributed, replicated system

upon which

various replication protocols with different guarantees/properties can be designed and implemented



Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial

FRED B. SCHNEIDER

Department of Computer Science, Cornell University, Ithaca, New York 14853

The state machine approach is a general method for implementing fault-tolerant services in distributed systems. This paper reviews the approach and describes protocols for two different failure models—Byzantine and fail stop. System reconfiguration techniques for removing faulty components and integrating repaired components are also discussed.

Categories and Subject Descriptors: C.2.4 **[Computer-Communication Networks]**: Distributed Systems—*network operating systems*; D.2.10 **[Software Engineering]**: Design—*methodologies*; D.4.5 **[Operating Systems]**: Reliability—*fault tolerance*; D.4.7 **[Operating Systems]**: Organization and Design—*interactive systems, real-time systems*

General Terms: Algorithms, Design, Reliability

Additional Key Words and Phrases: Client-server, distributed services, state machine approach

INTRODUCTION

Distributed software is often structured in terms of *clients* and *services*. Each service comprises one or more *servers* and exports *operations* that clients invoke by making *requests*. Although using a single, centralized, server is the simplest way to implement a service, the resulting service can only be as fault tolerant as the processor executing that server. If this level of fault tolerance is unacceptable, then multiple servers that fail independently must be used. Usually, replicas of a single server are executed on separate processors of a distributed system, and protocols are used to coordinate client interactions with these replicas. The physical and electrical isolation of processors in a distributed system ensures that server failures are independent, as required.

The *state machine approach* is a general method for implementing a fault-tolerant service by replicating servers and coordinating client interactions with server replicas.[1] The approach also provides a framework for understanding and designing replication management protocols. Many protocols that involve replication of data or software—be it for masking failures or simply to facilitate cooperation without centralized control—can be derived using the state machine approach. Although few of these protocols actually were obtained in this manner, viewing them in terms of state machines helps in understanding how and why they work.

This paper is a tutorial on the state machine approach. It describes the approach and its implementation for two representative environments. Small examples suffice to illustrate the points. However, the

[1] The term "state machine" is a poor one, but, nevertheless, is the one used in the literature.

Motivation:
Failures Happen Frequently & Everywhere

Technology

4 minute read · January 25, 2023 10:41 AM CST · Last Updated 20 days ago

Microsoft cloud outage hits users around the world

By Akriti Sharma

Media & Telecom

2 minute read · February 14, 2023 3:33 AM CST · Last Updated 13 hours ago

T-Mobile outage hits users across the U.S.

Reuters

TWITTER / TECH / WEB

Most people can tweet again, but Twitter still has issues

Illustration by Alex Castro / The Verge

/ For an hour and a half, many people were unable to post.

By MITCHELL CLARK
Updated Feb 8, 2023, 6:27 PM

TICK TOCK —

New data illustrates time's effect on hard drive failure rates

Backblaze examines 230,921 HDDs across 29 models from Seagate, Toshiba, and more.

SCHARON HARDING - 2/1/2023, 12:48 PM

Network, Storage, …
Any component may fail!

Fault-Tolerance :=

The property of a **system** continuing to **operate properly** in the event of **one or more faults** within some **components**

Today's Outline

Problem Model

- System Model
- Fault Model
- Correctness Model

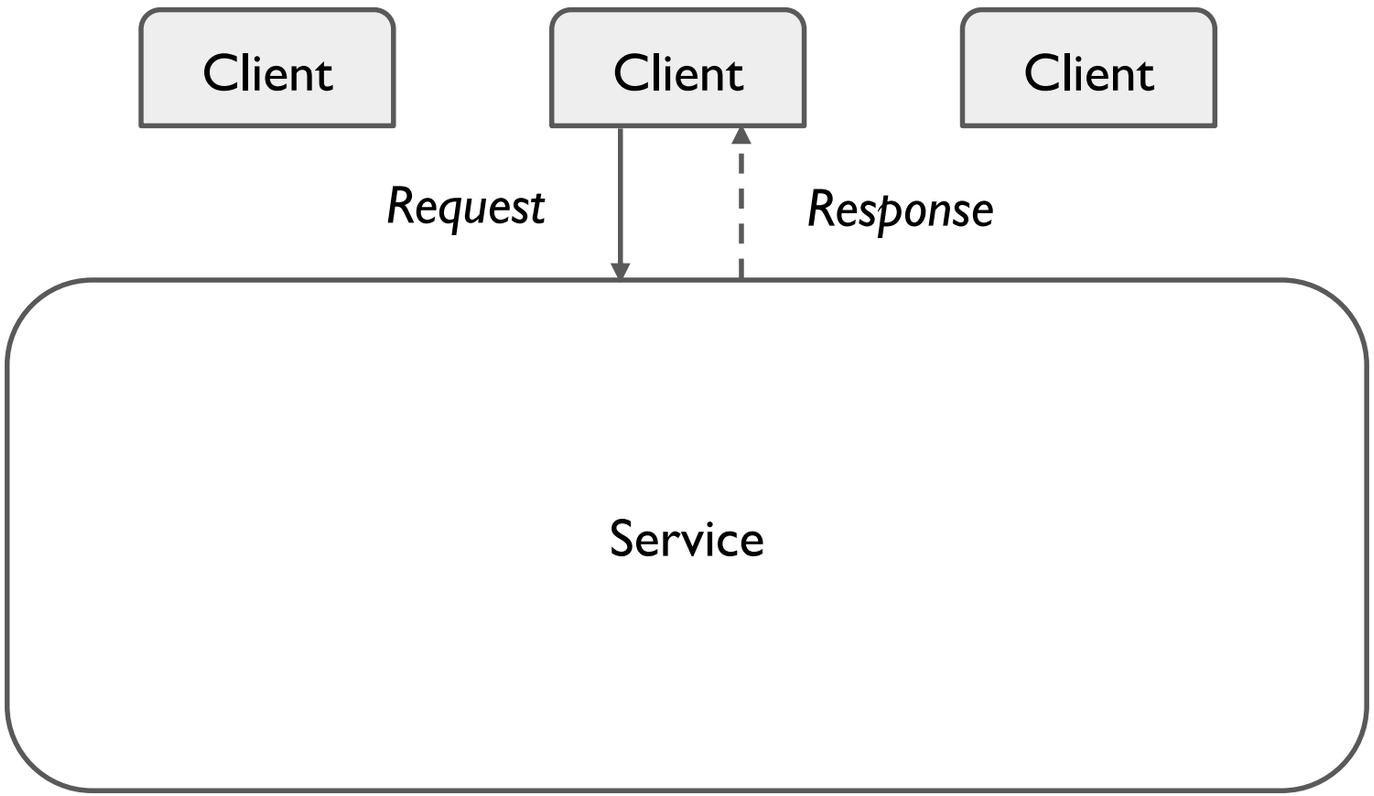
Approach: State Machine Replication (SMR)

- Program as Deterministic State Machines
- Replicating a Log of Commands

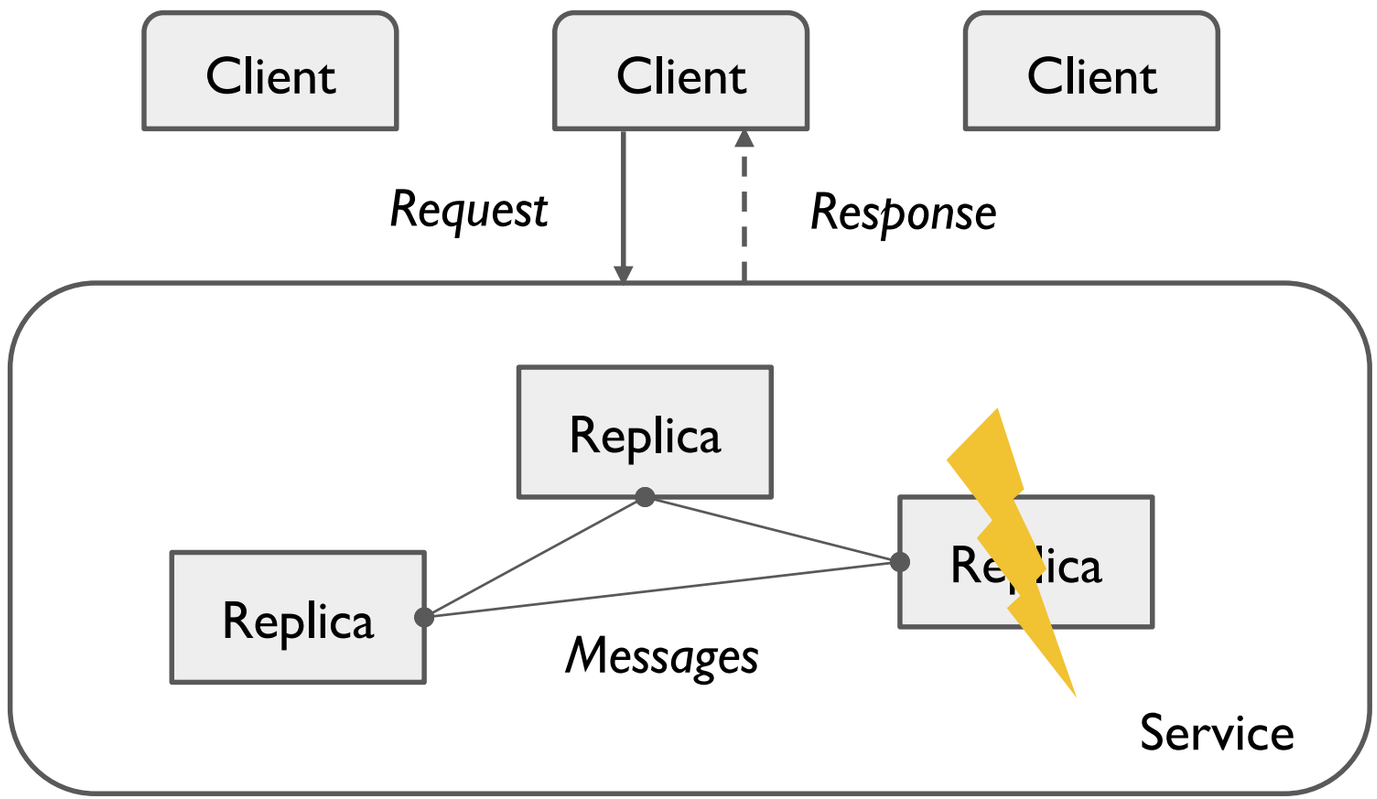
Fault-Tolerant SMR Technical Details

- Achieving Agreement & Ordering
- Weakening of Agreement & Ordering
- Faulty Output Devices or Clients

How is the system structured?



How is the system structured?



Replication

Keep multiple copies (that fail independently)

Appear as one service to clients

Reasons other than fault-tolerance to have replicas?

What types of faults occur?

The property of a **system** continuing to **operate properly** in the event of **one or more faults** within some **components**

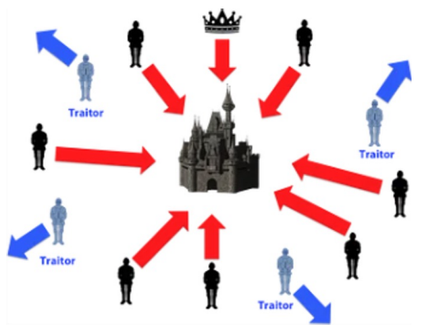
Non-Byzantine Failures

- **Fail-Stop**: Nodes may be down at any time and not respond to requests/messages, due to…
  - Power failures
  - Hardware failures
  - Disasters
  - Software crashes
- **Asynchronous Network**: Messages sent through the network may get…
  -
  -
  -

Byzantine Failures

Beyond fail-stop & asynchronous networks, the system could also have:

- **Malicious Nodes**: Some nodes may deliberately take incorrect actions, **such as?**
  -
  -
  -



How do we measure fault-tolerance?

Mean Time Between Failures (MTBF)

- Classic statistical metric
- Client-centric
- Typical in describing reliability of devices, e.g., hard drives and memory cards
- **Limitation** in describing reliability of a distributed service:
  Provides no insight for what is required from the system side for correct operation

Mean Time To Repair (MTTR)

- Often presented along with MTBF
- *P_avail* = MTBF / (MTBF + MTTR)

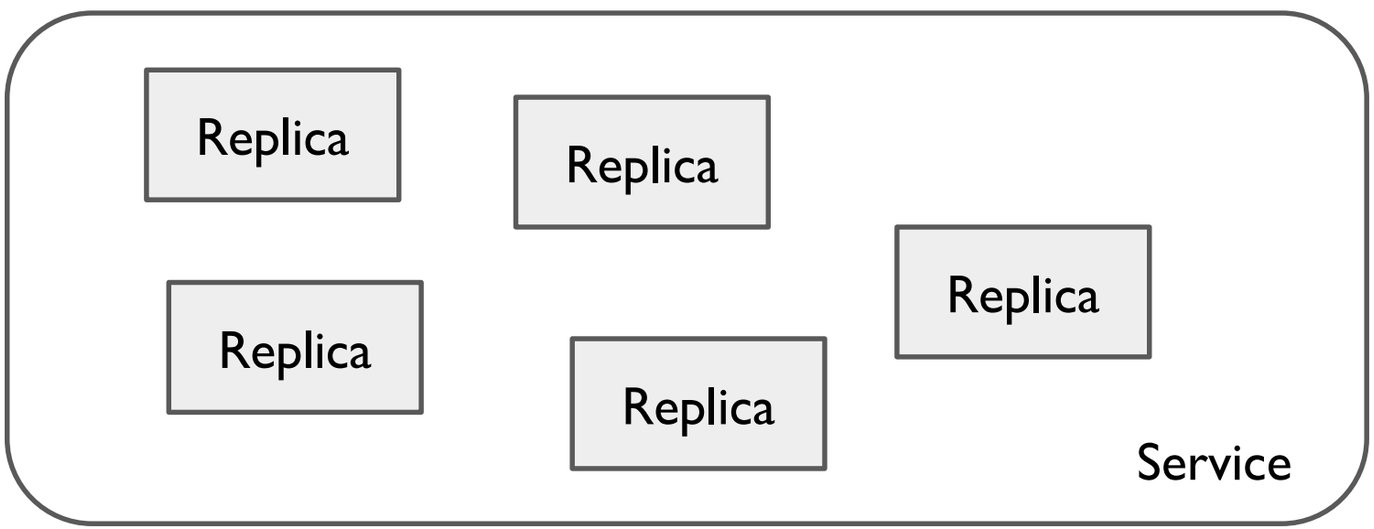
***t*-Fault-Tolerant**

- Number of replicas that can fail concurrently while system operates properly
- Implementation-centric
- **Assumptions?**
  -
  -
  -

To be t fault-tolerant, how many replicas?

Fail-Stop?

Byzantine?



What does "continue to operate properly" mean?

The property of a **system** continuing to **operate properly** in the event of **one or more faults** within some **components**

When up to certain bound of faults happen, the system should still be…

- **Responsive**: clients get timely responses for their requests
  - i.e., the service is still **available**
- **Consistent**: responses obey "semantic contract" between clients & service
  - Tricky problem!
  - Consistency levels:
    - Linearizability
    - Sequential consistency
    - Causal consistency
    - Eventual consistency
    - …

a.k.a. Strong Consistency:
Service appears as a *single*, *atomic* server that all clients talk to

State Machine Replication (SMR) or Replicated State Machine (RSM)

General framework for describing a distributed, replicated system, upon which various replication protocols with different guarantees/properties can be designed and implemented

An effective **approach** to designing & implementing
fault-tolerant distributed service through:

1) **modeling the program as state machines** and
2) **replicating a correctly-ordered log of client commands**

Program as (Deterministic) State Machine(s)

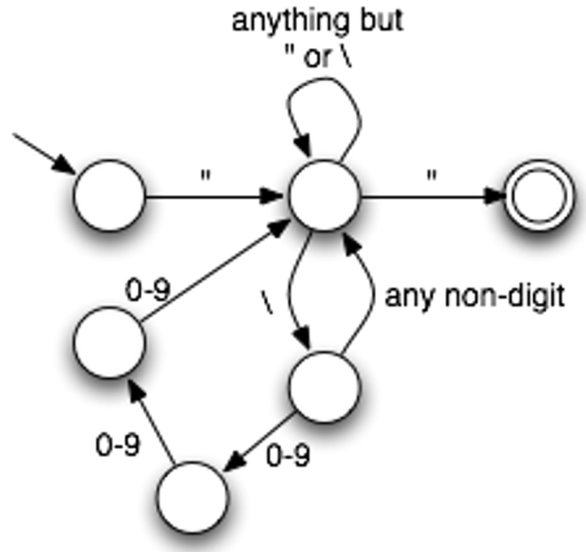
State Machine (SM) consists of…

State: State variables

- e.g., a data structure, such as a hash map of key-value pairs

Commands: Input operations from clients that?

- (possibly)
- (possibly)
- e.g., <put value “Room 1240” into key “CS 739 lecture location”>
- e.g., <get value for key “CS 739 lecture location”>




```
memory: state_machine
          var store: array[0 . . n] of word
    read: command(loc: 0 . . n)
          send store[loc] to client
          end read;
    write: command(loc: 0 . . n, value: word)
          store[loc] := value
          end write
    end memory
```

Other Assumptions

Single Sequence

State machine processes a serialized sequence of commands

- Each client request is a single command to specific SM
- Concurrent requests to same SM must be serialized properly

Deterministic

Outputs of state machine completely determined by sequence of requests it processes

- No independently random choices

Logical Time

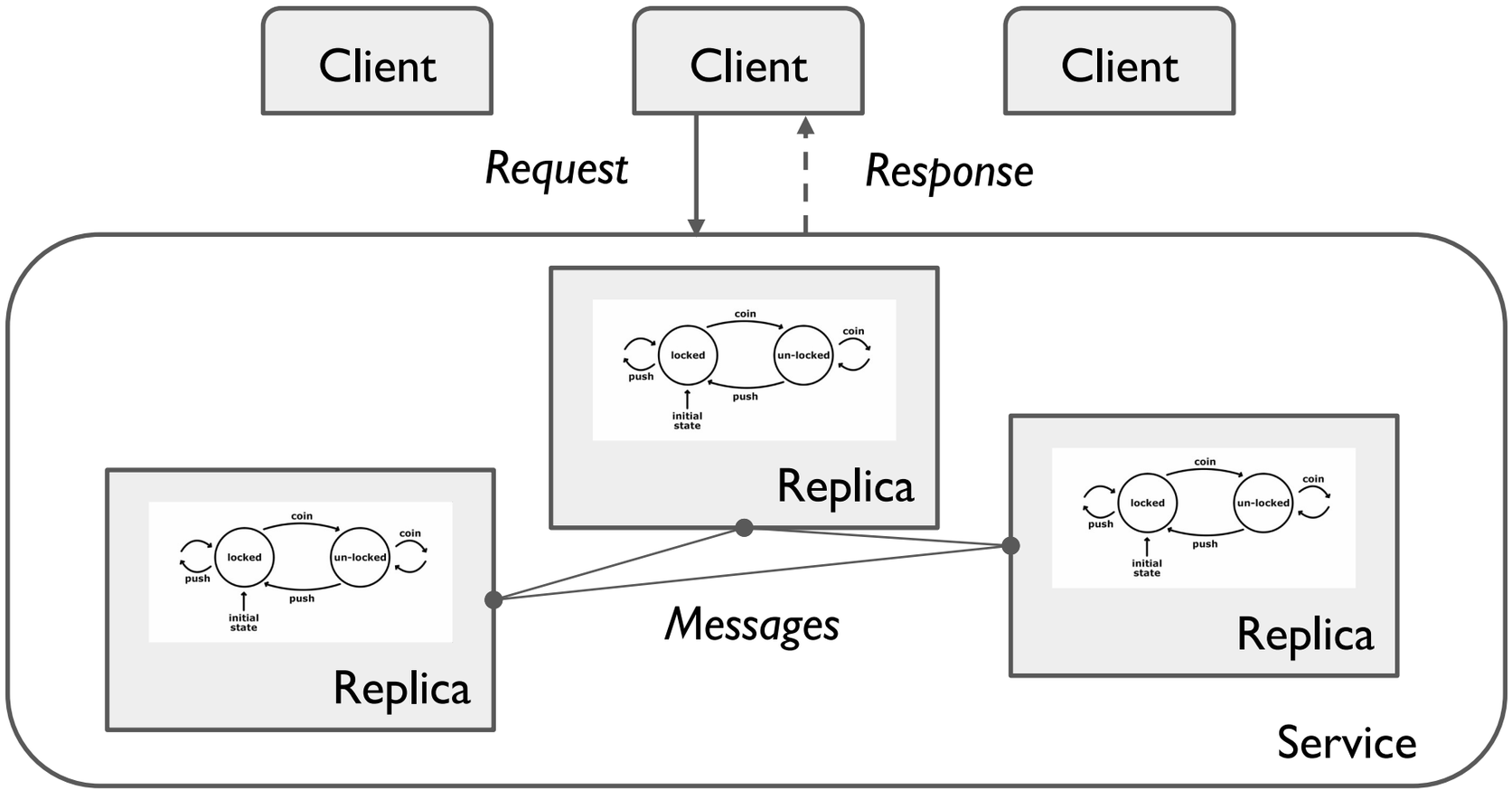
Requests & executions do not depend on physical time or irrelevant activities in system

- e.g., do not have real-time sensors as state
- Lamport's logical clock timestamps

State Machine Replication (SMR)

Send to all replicas?
Send to any replica?
Send only to leader?
Replica shares with all others?

a.k.a. Replicated State Machines (RSM)



State Machine Replication (SMR)

Typical SMR algorithm steps:

1. Client sends request (i.e. command to SM) to one server

2. Service **replicates input command** to (some number of) replicas

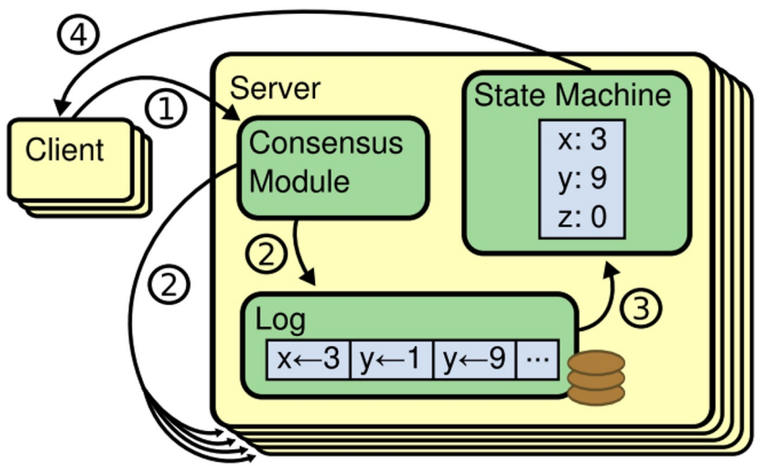
3. Replicas **apply (i.e. execute)** command on local SM copy
   - Since all replicas start from same initial state and SM is deterministic, resulting state and output is identical across replicas

4. Server (or separate observer) **materializes output** of command

What is being shared across replicas?

Share entire state?

- State can be very large
  (e.g., hash map of millions records)
- Communicating entire state among replicas is impractical



Replicate log of commands!

- **Why a good idea?**
  -
  -
- SM is deterministic and all copies start from same initial state
- **Replica Coordination:**
  Each replica receives and processes same sequence of events
  - **Agreement**:
  - **Order**:

How can all replicas Agree on commands?

Required Conditions

IC1: All non-faulty nodes agree on same value

IC2: If transmitter is non-faulty, all non-faulty nodes use its value

Use a *reliable broadcast* protocol:

- Designated entity acts as *transmitter*
  - May be client itself (sending request to all replicas it knows)
  - May be one of the replica servers (acting as *leader* of command)
    - **Pros and cons of leader?**
  -

- Transmitter broadcasts the request to other replicas

How to achieve Ordering?

High-level idea:
Obtain total ordering of requests tagged with unique identifiers

Decompose into two tasks…

- **Assignment** of identifiers:
  Must be unique across requests

- **Stability** of ordering:
  Request is **stable** at replica *i* once no request (from correct client) with lower identifier can be delivered later to *i*
  - Pick assignment scheme that provably passes a *stability test*

Achieving Ordering - #1
Logical Clock

Method #1: Lamport's **Logical Clock Timestamps**

T_p is integer counter maintained at processor *p*

- Incremented after each event at *p*
- Attached to each message sent by *p*
- Upon receipt of message with timestamp *t*, sets *T_p* to max(*T_p*, *t*) + 1

T_e is the timestamp of event *e* that occurs on *p*

- *T_e* := <*T_p*, *ID_p*>
- *T_e* is obviously *unique* for each event

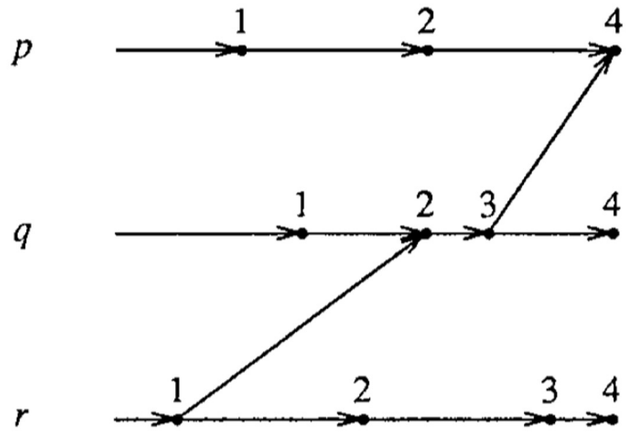


Figure 4. Logical clock example.

Achieving Ordering - #1
Logical Clock

What stability test can we use on *T_e*?

Assume:

- Communication channels are FIFO, and
- Replica *i* detects peer *j* has fail-stopped only after *i* has received the last message sent by *j*

Then:

- A request with timestamp *T_e* is stable at replica *i* if a request with larger *T_e'* has been received from every non-faulty peer replica

What about with an asynchronous network? (out of order, lost messages)

Tricky problem!

- Wait for future lectures on consensus & replication!

Achieving Ordering - #2
Synchronized Real-Time Clocks

Method #2: Rely on hardware-synchronized real-time clocks

If hardware guarantees no two replicas clocks deviate by more than δ secs, stable ordering is easier to achieve…

- Wait for sufficient number (Δ) of seconds, and everything with timestamps < T_p - Δ is stable
- Safe Δ derived from δ and properties of protocol

Achieving Ordering - #3
Replica-Generated Identifiers

Method #3: Let replicas generate a stable identifier for each command

Paper proposes a draft protocol similar to single-decree Paxos

- *Monotonic Proposal Numbers + Majority Voting*
- Details not in this lecture
- Wait for future lectures on consensus & replication!

Achieving Agreement & Ordering

Two requirements often discussed together

- Sometimes referred to as problem of Log-Based "Consensus"
- Achieving correct & efficient log replication is **core** of most distributed fault-tolerant replication protocols
- Words used interchangeably
  - State Machine Replication
  - Consensus Algorithms
  - Distributed Processors/Shared Objects
  - Consistent Agreement/Ordering/Atomic Broadcast

More exciting papers around this topic in future lectures!

When might not need Agreement or Ordering?

Faulty Output Devices

Can we use a voter to combine all outputs?

Solution depends on whether clients are inside or outside system

1) Faulty output used **outside** the system
   - **Solution?**
   -

2) Faulty replies to clients **inside** the system
   - **When can client proceed?** As soon as it receives output
   - Optimization if client co-located with a replica: no extra effort needed; Why?

Faulty Clients

What can go wrong if (one) client is faulty?

How to insulate RSM from faulty clients?

1. Replicate client; modify SM to handle N request copies
   - *Request buffering*: wait for enough copies of the same request
   - *Application-semantic-aware resolution (are they exactly identical? which to keep?)*
2. What if clients cannot be efficiently replicated?
   - *Defensive Programming:* Limit impact of faulty clients through *isolation* or *testing* mechanisms

Testing Example: *mutex* SM

release:
command
if *user* ≠ *client* → **skip**
□ *waiting* = Φ ∧ *user* = *client* →
user := Φ
□ *waiting* ≠ Φ ∧ *user* = *client* →
send *OK* **to** *head*(*waiting*);
user := *head*(*waiting*);
waiting := *tail*(*waiting*)
fi
end *release*

Isolation Example: *memory* SM
Restrict write requests from each client to certain isolated regions

Time-outs (to release resources)

Reconfiguration

How to handle more than t faults?

Why else good to remove fail-stop nodes from system?

Configuration management – Tricky

Configuration must be known to clients, SMRs, and output devices

- Change configuration request when detect failure or recovery
- New configurations must be scheduled for future so know before comes into effect
- Clients can query RSM for configuration changes

Conclusions

State Machine Approach

- Reduces replication to agreement on order
- Separates correctness from implementation
- Provides a template still used in modern systems

But in practice

- Determinism is hard
- Performance requires further innovation
- Real systems add layers

Summary

Replication = ordering + deterministic execution